

Utilizing Graph Neural Networks (GNN) with Deep Q-Learning with to Solve Dots and Boxes

Kevin Chen

Systems Lab
Yilmaz 2026

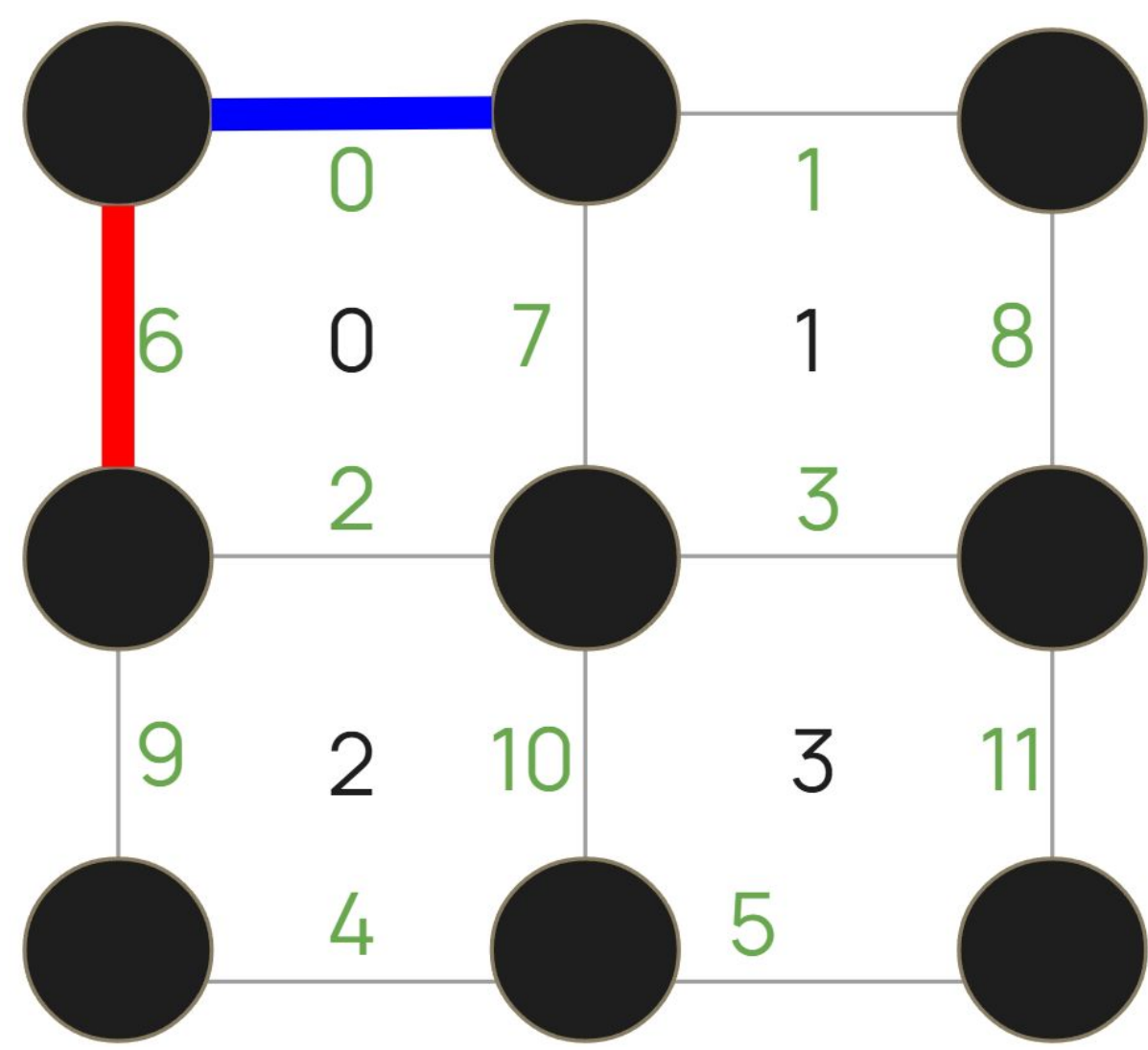


Figure 1: Indexing of the Dots and Boxes Board representation

Introduction

Dots and Boxes a deterministic game where each player takes turns drawing a line connecting dots on a grid. A annotated example can bee seen in Figure 3. If a player creates a box, they get to go again. Each player wants to maximize the number of boxes they create. My proposed solution uses a GNN in combination with Deep Q-Learning to create an agent to solve the game.

Novelty + Impact

Previous solutions include algorithms utilizing heuristics, Alpha Beta Pruning, Monte Carlo Search Trees, and Deep-Q Learning with Convolutional Neural Networks (CNN). However, the novelty that comes with GNNs is the evaluation of the problem through a graph structure. Additionally, it should decrease computational load for q-values in reinforcement learning, without a significant sacrifice in model performance. The model was able to generalize gameplay to larger boards.

Algorithm 1 Deep Q-Learning Algorithm^[3]

- 1: Initialize replay memory D to capacity N
- 2: Initialize action-value function Q with weights θ
- 3: Initialize target action-value function $\bar{Q}$ with weights $\theta^- = \theta$
- 4: **for** episode = 1, M **do**
- 5: Initialize initial game state $s_1 = \{x_1\}$
- 6: **for** $t = 1, T$ **do**
- 7: With probability ϵ select a random action a_t
- 8: otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t, a; \theta))$
- 9: Execute action a_t in emulator and observe reward
- 10: Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
- 11: Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
- 12: Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
- 13: Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \bar{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$
- 14: Perform a gradient decent step on $(y_j - Q(\phi_{j+1}, a; \theta^-))^2$ with respect to the network parameters θ
- 15: **end for**
- 16: Every C episodes reset $\bar{Q} = Q$
- 17: **end for**

Figure 2: The deep q-learning algorithm

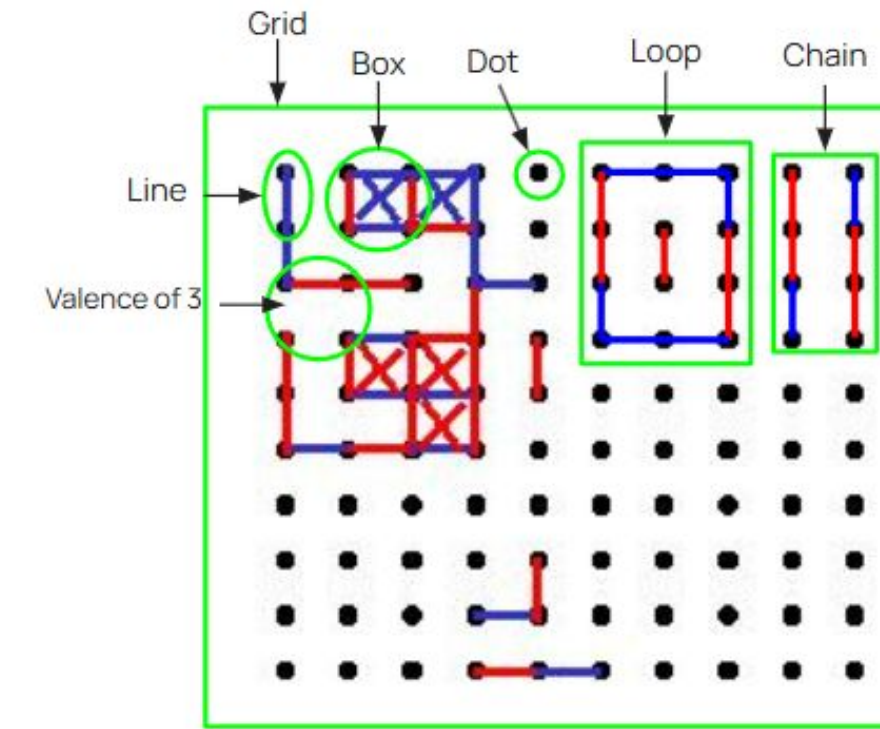


Figure 3: Labeled game of Dots and Boxes

Opponent	Win Rate	Boxes Captured	Time per move for GNN (s)	Time per move for opponent (s)
Alpha Beta of depth 3 (3x3)	71%	60.7%	0.0563	0.0218
Alpha Beta of depth 3 (4x4)	86%	72.56%	0.0544	0.0925
Alpha Beta of depth 3 (4x4)	88%	73.20%	0.0569	6.866
Random (3x3)	72%	59.22%	0.0567	5.62e-6
Random (4x4)	83%	72.31%	0.0785	1.406e-5
Random (5x5)	85%	70.84%	0.0995	8.454e-6
Heuristics (3x3)	71%	59.56%	0.0544	2.177e-5
Heuristics (4x4)	83%	70.69%	0.0538	3.767e-5
Heuristics (5x5)	94%	77.52%	0.0531	5.167e-5

Figure 4: Results of 1 vs. 1 of GNN model against other models in 100 games with the win rate, boxes captured, time per move, and time per move for the opponent.

Methods

The board was transformed into a graph where the nodes represent lines, and the edges represent shared boxes, as seen in Figure 1. Then the GNN was created using the Keras library and the message passing was implemented through matrix multiplication: $H = \sigma(A \times N \times W)$, where σ is the activation function (ReLU), A is the adjacency matrix, N is the node feature matrix, W is the weights, which results in a sum based aggregation. Then, this network structure is passed through the algorithm as seen in Figure 2, which outputs a GNN that maps state-action pairs to q-values. Then the GNN is trained through the program in Figure 5. It then is used to play games against other models, specifically the random agent, a heuristic model, and an Alpha Beta model.

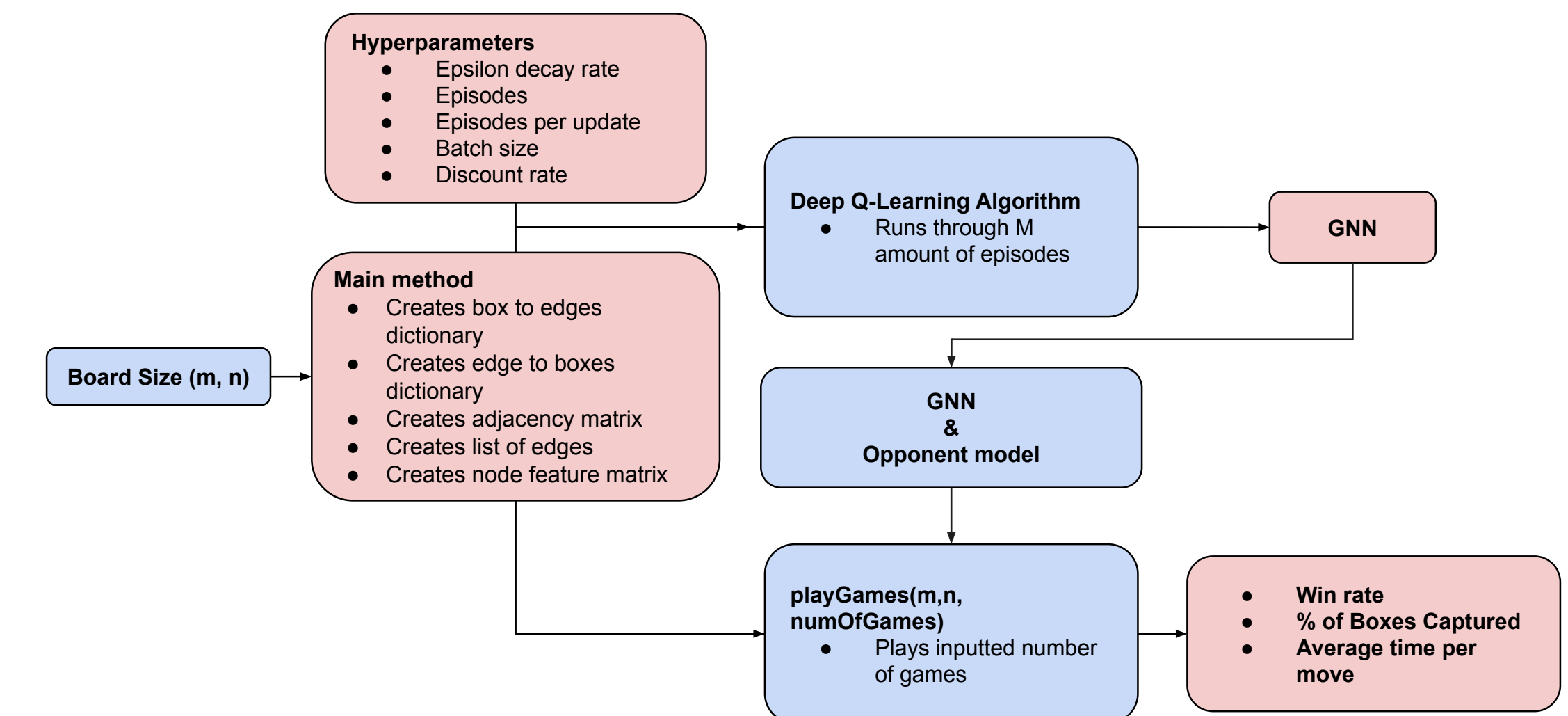


Figure 5: Program outline

Results

The model was tested against a variety of bots and board sizes and the results are depicted in Figure 4. My model achieved a win rate average of 81% across all experiments, and an average of 68.49% boxes captured, with the best performance against the heuristic model on a 5x5 board. It also achieved a superior time complexity, where increasing the board size had a negligible effect on the time to compute a move. The performance of the Deep Q-Learning model with a GNN is inversely related to the size of the board.

Future Work

In the future, I would like add the ability to train on different size boards in the same session. Additionally, I want to play the model against previous versions of itself in order to attain better generalization. This could all likely improve the win rate against other models. I would like to add more agents to test my model and utilize other versions of GNNs such as Graph Attention Networks.